SkillChef — Team Task Breakdown

A 4-person split of the SkillChef project. Each person owns a feature domain plus a shared cross-cutting responsibility the other three depend on.

Person 1 — Auth & User System

Database & models

Design the USERS, PROFILES, and FOLLOWS tables exactly as in the ER diagram (uuid PKs, unique email, password_hash, avatar_url, global_xp, level, created_at, bio, preferences as json, composite PK on follows). Set up the PostgreSQL instance on Render, write the initial migration system (Prisma or Drizzle), and become the gatekeeper for the master schema — everyone's PRs that touch DB go through you so foreign keys, indexes, and naming stay consistent.

Auth Service

Signup, login, logout, JWT issuance and refresh, password hashing with bcrypt/argon2, email verification, password reset flow. Middleware for protected routes that the other three will import.

User Service

Profile CRUD, avatar upload endpoint (calling Person 4's storage layer), preferences (notifications, theme, language), public profile view, global_xp and level read endpoints.

Social Service

Follow/unfollow, followers list, following list, "is following" checks, follower counts. The follows feed query that Person 4 will consume for the community feed.

Infrastructure

Buy/configure the domain, set up Cloudflare DNS, SSL/TLS certificates on Render, DDoS protection rules, CORS config for the Next.js frontend.

Person 2 — Skill Tree & Learning Content

Database & models

SKILL_DOMAINS, SKILL_NODES (with tier and xp_reward), SKILL_PROGRESS, LESSONS, QUIZZES, XP_TRANSACTIONS. Seed the database with the initial skill tree content (domains like "Baking", nodes like "Knife Skills" → "Julienne", lessons, quizzes).

Skill Tree Service

Endpoints for fetching the full tree for a user (with their progress overlaid), unlocking nodes based on tier prerequisites, marking nodes complete, computing progress_percent.

Lesson Service

Lesson CRUD for admins, lesson playback endpoint (returns video_url from Person 3's video hosting), duration tracking, marking lessons watched.

Quiz Service

Quiz delivery (without the correct_answer field exposed), answer submission, scoring, awarding XP via XP_TRANSACTIONS, updating SKILL_PROGRESS and USERS.global_xp + level on pass.

Progression Service

XP-to-level formula, level-up events, leaderboard query (top users by global_xp), XP transaction history.

Frontend skeleton

Set up the Next.js 14 app router project, Tailwind config, shared layout (nav, sidebar, footer), auth context wrapper around Person 1's JWT, a component library (Button, Card, Modal, Input, Avatar, Skeleton loaders), and the skill tree visualization page itself. The other three build their pages inside this skeleton.

Caching

Redis setup on Render, cache the skill tree response per user, cache session tokens, set up cache invalidation when progress changes.

Person 3 — Challenges & AI Assistant

Database & models

CHALLENGES (title, type, xp_reward, start_date, end_date), SUBMISSIONS (media_url, ai_feedback, score), AI_CONVERSATIONS, AI_MESSAGES (role, content).

Challenge Service

Challenge CRUD for admins, list active challenges (filtered by start_date/end_date), challenge detail, user's submission history per challenge.

Submission Service

Submission upload endpoint (media goes through Person 4's storage), trigger AI analysis job, return score and ai_feedback, award XP through Person 2's XP transaction system on successful submission.

AI Service

AI conversation CRUD, message history pagination, streaming chat endpoint for the AI Assistant feature, prompt templates for "rate this dish" vs "answer cooking question" vs "suggest next skill," token/cost tracking per user.

External integrations

OpenAI or Anthropic API client with retry and rate-limit handling, Mux or Cloudflare Stream integration for lesson and submission videos (upload, transcode, signed playback URLs), SendGrid for transactional email (welcome, password reset for Person 1, challenge-ending reminders, weekly digest).

Background Worker

BullMQ or similar on top of Person 2's Redis. Jobs for: processing submissions (transcode video → run AI vision analysis → score → notify user), sending scheduled emails, recomputing challenge leaderboards, cleaning up expired challenges. Retry logic and dead-letter handling.

Frontend pages

Challenges list and detail, submission upload UI, AI Assistant chat interface.

Person 4 — Community Feed

Database & models

RECIPE_POSTS (title, description, image_url), COMMENTS, LIKES. Indexes for feed queries (created_at, user_id, recipe_post_id).

Recipe Service

Create/edit/delete recipe posts, image upload through your own storage layer, recipe detail endpoint, user's recipe list.

Feed Service

The main community feed query — pulling recipes from people the user follows (joining on Person 1's FOLLOWS table), pagination, sort by recency, "trending" sort by like count in the last 24h.

Engagement Service

Like/unlike a post, comment CRUD, like counts, comment counts, recent commenters preview.

Notifications

When someone likes or comments on your post, follows you (with Person 1), or replies to your comment. In-app notification model, mark-as-read endpoints. Push to SendGrid for email notifications (using Person 3's email setup) for users who opted in via Person 1's preferences.

Storage layer

This is the shared piece everyone depends on. Set up S3 (or Cloudflare R2) bucket, write a unified upload service — signed upload URLs, file type and size validation, image resizing/optimization pipeline, CDN URLs. Export a clean SDK that Person 1 uses for avatars, Person 2 for any lesson thumbnails, Person 3 for submission media, and you for recipe images.

DevOps & monitoring

Render deployment configs (web service, worker service, cron jobs), environment variable management across dev/staging/prod, auto-deploy from main branch, Pingdom uptime checks, Sentry error tracking wired into both frontend and backend, Logtail for structured logging, PostHog for product analytics events.

Frontend pages

Community feed, recipe detail page, recipe creation page, notifications panel.